

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

MODUL 17

SKEMA PEMROSESAN SEKUENSIAL



Disusun oleh:

MANGGALA PATRA RADITYA

109082500179

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhli

Renisa Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

MODUL 17

1. Soal 1

Source code

```
package main

import "fmt"

func main() {

    var angka float64

    var jumlah float64

    var banyakData int

    fmt.Println("Masukkan bilangan (akhiri dengan 9999):")

    for {

        fmt.Scan(&angka)

        if angka == 9999 {

            break

        }

        jumlah += angka

        banyakData++

    }

    if banyakData > 0 {
```

```

        rataRata := jumlah / float64(banyakData)

        fmt.Printf("Rata-rata = %.2f\n", rataRata)

    } else {

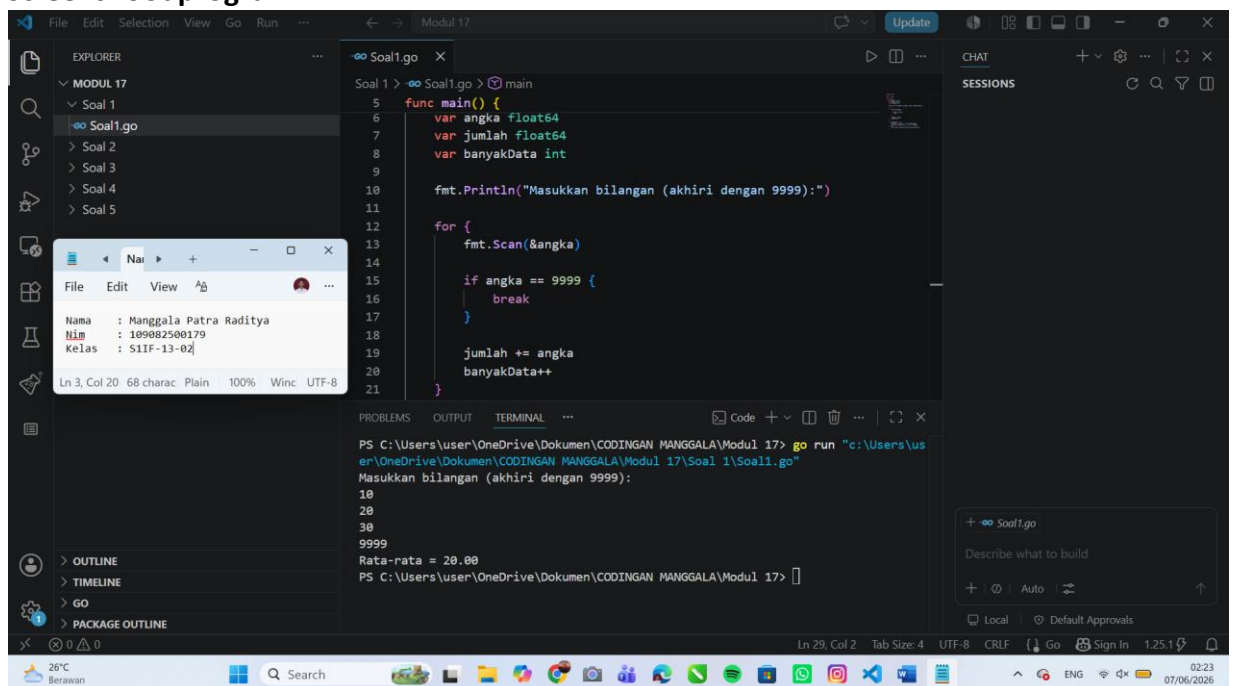
        fmt.Println("Tidak ada data yang dimasukkan.")

    }

}

```

Screenshoot program



Deskripsi program

Program membaca sejumlah bilangan real hingga ditemukan **marker 9999**. Nilai 9999 tidak ikut dihitung. Setelah semua data dibaca, program menghitung dan menampilkan **rata-rata** dari bilangan yang dimasukkan.

2. Soal 2

Source code

```
package main

import "fmt"

func main() {

    var x string

    var n int

    fmt.Scan(&x)

    fmt.Scan(&n)

    jumlahKemunculan := 0

    posisiPertama := -1

    for i := 0; i < n; i++ {

        var data string

        fmt.Scan(&data)

        if data == x {

            jumlahKemunculan++

            if posisiPertama == -1 {

                posisiPertama = i + 1

            }

        }

    }

}
```

```
        }

    }

}

if jumlahKemunculan > 0 {

    fmt.Println("String ditemukan")

} else {

    fmt.Println("String tidak ditemukan")

}

if posisiPertama != -1 {

    fmt.Println(posisiPertama)

} else {

    fmt.Println("Tidak ditemukan")

}

fmt.Println(jumlahKemunculan)

if jumlahKemunculan >= 2 {

    fmt.Println("Ya")

} else {

    fmt.Println("Tidak")

}

}
```

Screenshoot program

The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a Go program named `Soal2.go` that implements a search and counting algorithm. The terminal shows the output of the program, which includes the input strings and the results of the search.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x string
7     var n int
8
9     fmt.Scan(&x)
10    fmt.Scan(&n)
11
12    jumlahKemunculan := 0
13    posisiPertama := -1
14
15    for i := 0; i < n; i++ {
16        var data string
17        fmt.Scan(&data)
```

The terminal output shows the following sequence of inputs and results:

```
apel
5
jeruk
apel
mangga
apel
pisang
String ditemukan
2
2
Ya
```

Deskripsi program

Program ini digunakan untuk mencari sebuah string **x** di dalam kumpulan **n** string yang diinputkan pengguna. Program akan memeriksa apakah string **x** terdapat dalam kumpulan data, menentukan posisi pertama kemunculannya, menghitung jumlah kemunculannya, serta memeriksa apakah string **x** muncul sedikitnya dua kali. Proses pencarian dilakukan dengan membandingkan setiap string yang diinput dengan string **x** menggunakan perulangan hingga seluruh data selesai diperiksa.

3. Soal 3

Source code

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    var n int

    fmt.Scan(&n)

    rand.Seed(time.Now().UnixNano())

    var A, B, C, D int

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            A++
        }
    }
}
```



```
        } else if x >= 0.5 && y < 0.5 {

            B++

        } else if x >= 0.5 && y >= 0.5 {

            C++

        } else {

            D++

        }

    }

    fmt.Printf("Curah hujan daerah A: %.4f\n", float64(A)*0.0001)

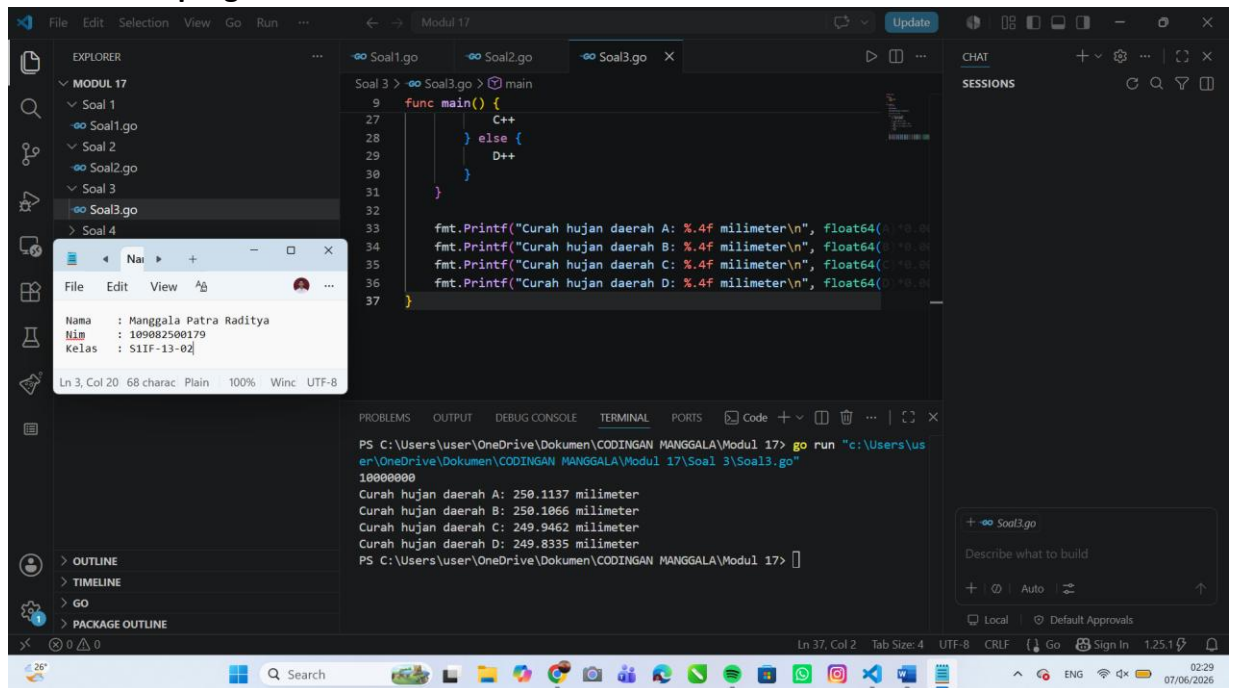
    fmt.Printf("Curah hujan daerah B: %.4f\n", float64(B)*0.0001)

    fmt.Printf("Curah hujan daerah C: %.4f\n", float64(C)*0.0001)

    fmt.Printf("Curah hujan daerah D: %.4f\n", float64(D)*0.0001)

}
```

Screenshoot program



```
Soal3.go
9 func main() {
27     // C++
28     // } else {
29     //     D++
30     // }
31 }
32
33 fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(A)*0.0001)
34 fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(B)*0.0001)
35 fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(C)*0.0001)
36 fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(D)*0.0001)
37 }
```

PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 17> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 17\Soal 3\Soal3.go"

10000000

Curah hujan daerah A: 250.1137 milimeter
Curah hujan daerah B: 250.1066 milimeter
Curah hujan daerah C: 249.9462 milimeter
Curah hujan daerah D: 249.8335 milimeter

Deskripsi program

Program ini digunakan untuk mensimulasikan pengukuran curah hujan pada empat daerah, yaitu A, B, C, dan D. Pengguna memasukkan jumlah tetesan air hujan yang akan disimulasikan, kemudian program membangkitkan koordinat acak (x , y) pada bidang berukuran 1×1 menggunakan fungsi `rand.Float64()`. Setiap tetesan akan diklasifikasikan ke salah satu daerah berdasarkan posisinya. Setelah seluruh tetesan diproses, program menghitung banyaknya tetesan yang jatuh pada masing-masing daerah dan mengonversikannya menjadi curah hujan dengan ketentuan satu tetesan setara dengan **0,0001 milimeter**. Hasil akhirnya adalah curah hujan untuk daerah A, B, C, dan D dalam satuan milimeter.

4. Soal 4

Source code

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var n int
    fmt.Scan(&n)

    var jumlah float64
    var piLama, piBaru float64
    var ketemu bool
    var indeks int

    for i := 1; i <= n; i++ {
        suku := 1.0 / float64(2*i-1)

        if i%2 == 0 {
            suku = -suku
        }
    }
}
```

```

        jumlah += suku

        piBaru = 4 * jumlah

        if i > 1 {
            if math.Abs(piBaru-piLama) < 0.00001 &&
!ketemu {

                ketemu = true

                indeks = i

                fmt.Printf("Hasil PI: %.9f\n",
piLama)

                fmt.Printf("Hasil PI: %.9f\n",
piBaru)

                fmt.Printf("Pada i ke: %d\n",
indeks)

            }

        }

        piLama = piBaru

    }

    if !ketemu {

        fmt.Printf("Hasil PI: %.9f\n", piBaru)

    }

}

```

Screenshoot program

The screenshot shows a Go IDE with a project named 'MODUL 17'. The file explorer on the left shows a directory structure with files 'Soal1.go', 'Soal2.go', 'Soal3.go', and 'Soal4.go'. The main editor displays the code for 'Soal4.go', which is a Go program for calculating pi using the Leibniz series. The code includes imports for 'fmt' and 'math', a 'main' function, and a loop that calculates the sum of the series. The terminal at the bottom shows the output of the program, which includes the calculated value of pi and the index of the first term that meets the condition.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     var n int
10    fmt.Scan(&n)
11
12    var jumlah float64
13    var pilama, piBaru float64
14    var ketemu bool
15    var indeks int
16
17    for i := 1; i <= n; i++ {
```

Terminal output:

```
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 17> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 17\Soal 4\Soal4.go"
1000000
Hasil PI: 3.141587654
Hasil PI: 3.141597654
Pada i ke: 200001
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 17>
```

Deskripsi program

Program ini digunakan untuk menghitung nilai π menggunakan deret Leibniz, yaitu deret harmonik berganti tanda yang nilainya mendekati $\pi/4$. Pengguna memasukkan jumlah suku pertama yang akan dihitung, kemudian program menjumlahkan setiap suku secara berurutan dengan tanda positif dan negatif secara bergantian. Hasil penjumlahan dikalikan 4 untuk memperoleh pendekatan nilai π . Selain itu, program membandingkan dua nilai π yang diperoleh dari iterasi berturut-turut dan menghentikan pencarian ketika selisih keduanya kurang dari 0,00001, kemudian menampilkan kedua nilai π tersebut beserta indeks suku saat kondisi tersebut pertama kali terpenuhi.

5. Soal 5

Source code

```
package main

import (
    "bufio"
    "fmt"
    "math/rand"
    "os"
    "strconv"
    "strings"
    "time"
)

func main() {
    scanner := bufio.NewScanner(os.Stdin)

    rand.Seed(time.Now().UnixNano())

    for scanner.Scan() {
        line := strings.TrimSpace(scanner.Text())

        if line == "" {
            continue
        }
    }
}
```

```
parts := strings.Split(line, ":")

if len(parts) != 2 {
    continue
}

nStr := strings.TrimSpace(parts[1])
n, err := strconv.Atoi(nStr)
if err != nil {
    continue
}

inside := 0

for i := 0; i < n; i++ {
    x := rand.Float64()*2 - 1
    y := rand.Float64()*2 - 1

    if x*x+y*y <= 1 {
        inside++
    }
}

pi := 4.0 * float64(inside) / float64(n)
```

```

        fmt.Println("Banyak Topping:", n)

        fmt.Println("Topping pada Pizza:", inside)

        fmt.Printf("PI : %.10f\n", pi)

        fmt.Println()

    }

}

```

Screenshoot program

The screenshot shows a Go IDE with the following components:

- EXPLORER:** Shows a project structure with 'MODUL 17' containing 'Soal 1' through 'Soal 4'.
- EDITOR:** Displays the code for 'Soal5.go', which implements a Monte Carlo simulation for estimating π . The code uses a scanner to read input, calculates the area of a circle and a square, and estimates π based on the ratio of points inside the circle to the total points.
- TERMINAL:** Shows the output of the program for three different input values (10, 256, and 5000). The output includes the number of points ('Banyak Topping'), the number of points inside the circle ('Topping pada Pizza'), and the estimated value of π .

```

Soal 5 > Soal5.go > main
12
13 func main() {
14     scanner := bufio.NewScanner(os.Stdin)
15
16     rand.Seed(time.Now().UnixNano())
17
18     for scanner.Scan() {
19         line := strings.TrimSpace(scanner.Text())
20         if line == "" {
21             continue
22         }
23     }
24
25     PI : 3.1401746523
26
27     Banyak Topping: 10
28     Banyak Topping: 10
29     Topping pada Pizza: 9
30     PI : 3.6000000000
31
32     Banyak Topping: 256
33     Banyak Topping: 256
34     Topping pada Pizza: 198
35     PI : 3.0937500000
36
37     Banyak Topping: 5000
38     Banyak Topping: 5000
39     Topping pada Pizza: 3944
40     PI : 3.1552000000

```

Deskripsi program

Program ini mengimplementasikan metode Monte Carlo untuk mengestimasi nilai π (phi) berdasarkan simulasi penyebaran titik secara acak pada sebuah bidang persegi yang memuat lingkaran (pizza). Setiap titik yang dihasilkan secara acak diperiksa apakah berada di dalam lingkaran atau tidak, kemudian dihitung jumlah titik yang masuk sebagai “topping pada pizza”. Dengan membandingkan jumlah titik di dalam lingkaran terhadap total titik yang diinputkan, program menghitung pendekatan nilai π berdasarkan perbandingan luas lingkaran dan persegi, di mana semakin besar jumlah titik yang digunakan, semakin akurat hasil estimasi yang diperoleh.